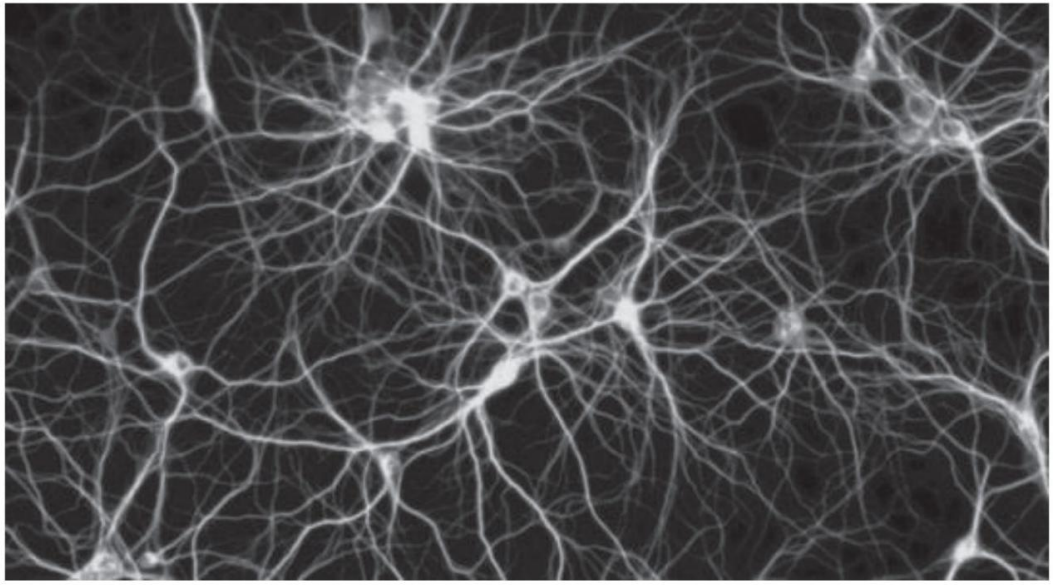
Machine Translated by Google

[Chap 11] Artificial Neural Network Basics

Machine Translated by Google

Overview of Artificial Neural Networks

- Artificial Neural Network (ANN)
  - Also abbreviated as Neural Network (NN) – A model that implements intelligence by mimicking the neural networks of animals



[그림 11-2] 신경망의 구조(자료: The Conversation, 2017)[3]

Machine Translated by Google

Basic unit of a neural network: neuron

- Receives signals from other neurons via dendrites, passes through the axon, and transmits signals to other neurons connected to the axon terminal.

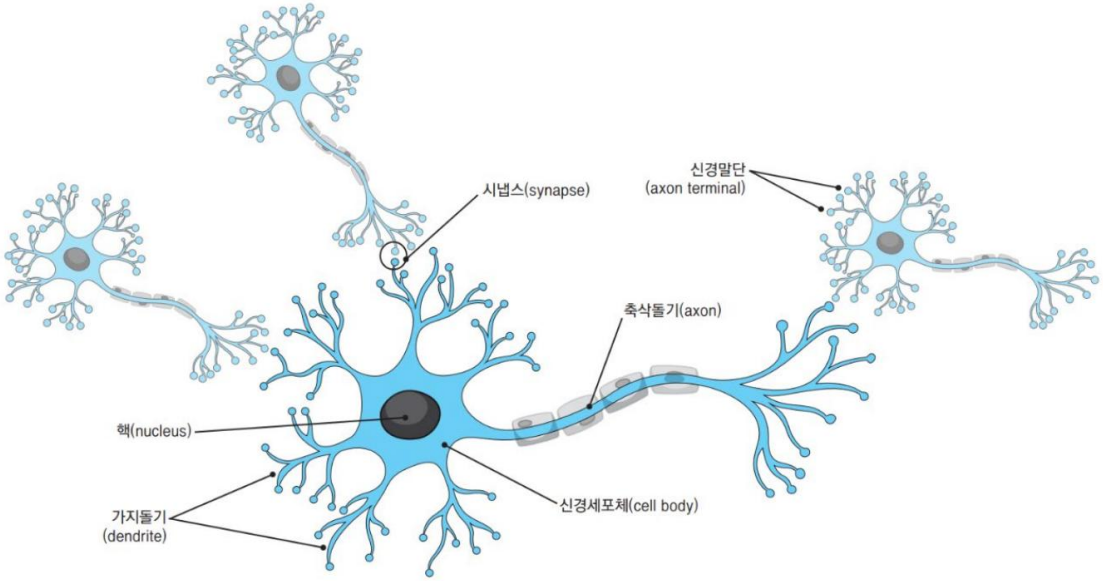
The point where neurons are connected is called a synapse.

– Sensitivity to input stimuli depending on the concentration of neurotransmitters in the synapse change

– When the total amount of stimulation exceeds a threshold, it transmits the stimulation to other neurons.

Intelligence manifests as neurons are connected like a network

– The human brain contains approximately 100 billion neurons and 100 trillion synapses.



[그림 11-1] 뉴런의 구조

Machine Translated by Google

Basic mechanisms of neurons

- Compare the weighted sum of input stimuli with the threshold

```
sum = 0
for i in range(0, n): sum += w[i]*x[i]
if sum>=threshold : transmit_to_other_neurons()
```

– n: Number of neurons connected to the input synapse of the dendrite

– x: All input values received from input synapses

– w: Weight for each input synapse

– Calculate the weighted sum of w[i] and x[i] for all input synapses, and this value
When a specific threshold is exceeded, the signal propagates to other connected neurons.

- In the code, signal propagation is represented by a virtual function called transmit_to_other_neurons()

– If the threshold is not exceeded, the signal is ignored

Machine Translated by Google

History of Artificial Neural Networks

- Perceptron

  – Published by Frank Rosenblatt in 1957 – Computer implementation of neuron operation,

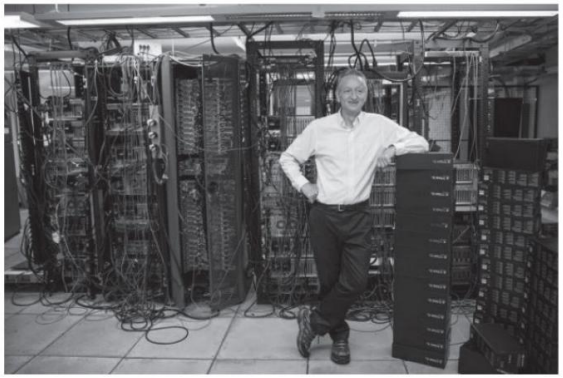
  performs simple classification

[그림 11-3] 퍼셉트론 부품으로 연구중인 프랭크 로젠블라트(왼쪽) (자료: Cornell Chronicle, 2019)

- AI Winter – Marvin

  Minsky pointed out the lack of scalability of the perceptron

  – Loss of interest from the public and academia



[그림 11-4] 딥러닝의 대부 제프리 힌튼 (자료: U of T news, 2018)[9]

- Backpropagation algorithm – introduced by Geoffrey

  Hinton – theoretically enabled the

  learning of deep neural networks, but,

  In reality, the number of layers is limited.

- Deep Learning – Geoffrey

  Hinton (2006), “A Fast Learning Algorithm for Deep Belief Nets” – Presented

  an efficient method for training deep neural networks

Machine Translated by Google

Perceptron model

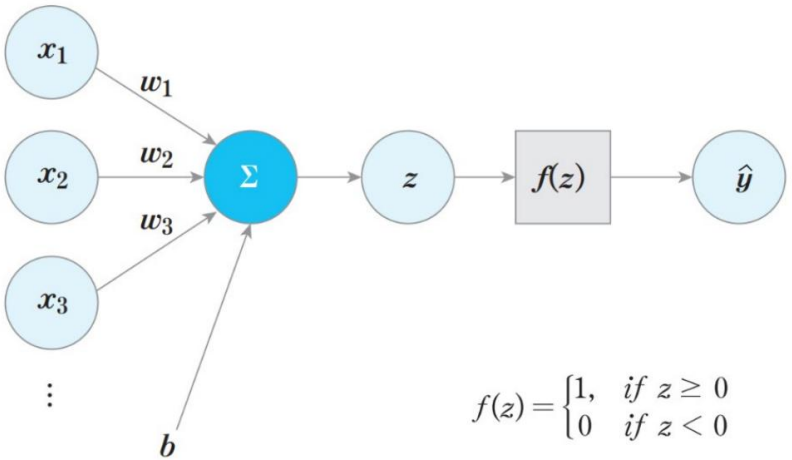
- The first artificial neural network model
  - Exactly mimics the way neurons work

- Calculate weighted sum by multiplying 1, 2, 3) input values (1, 2, 3) by weights (
  - = 1 1 + 2 2 + 3 3 + ÿ +

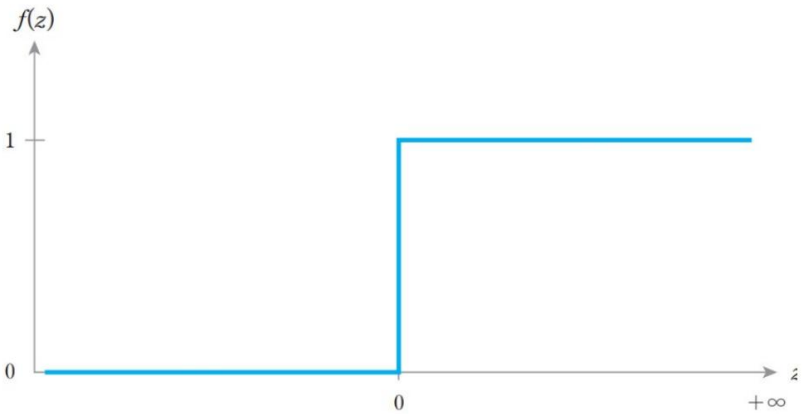
- Output 1 if the weighted sum is 0 or greater, and 0 if it is less than 0.

$$\hat{y} = f(z) = \begin{cases} 1, & \text{if } z \geqq 0 \\ 0, & \text{if } z < 0 \end{cases}$$

  - ( ): activation function



[그림 11-5] 퍼셉트론의 구조



[그림 11-6] 퍼셉트론의 활성화 함수

Machine Translated by Google

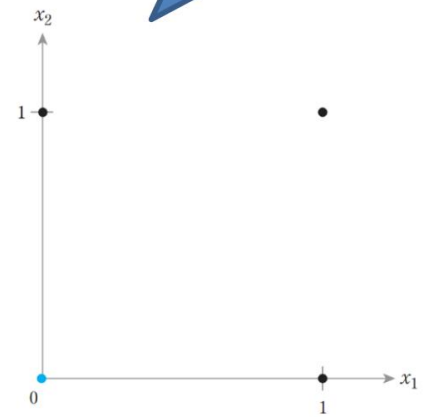
Perceptron Example

- Example data: or logical operation

〈표 11-1〉 퍼셉트론 예제 데이터

입력값		정답값
x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

Answer value 0 is blue, 1 is a black dot (•) Display as



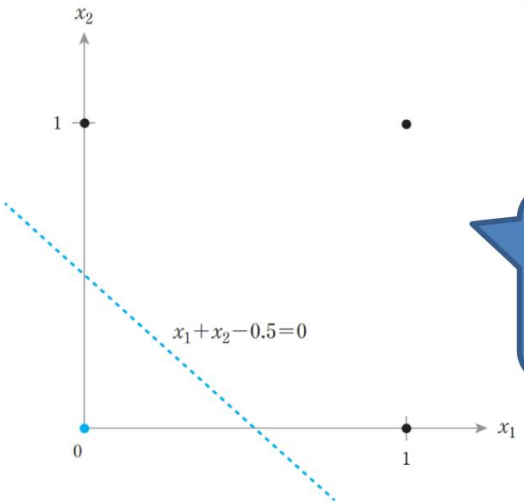
퍼셉트론 예제 데이터의 그래프

- Goal Classification

$$w_1x_1 + w_2x_2 + b < 0, \quad \text{if } x_1 = x_2 = 0$$
$$w_1x_1 + w_2x_2 + b \geqq 0, \quad otherwise$$

- Examples of values satisfying the above equation

– 1 = 1, 2 = 1, = ÿ0.5



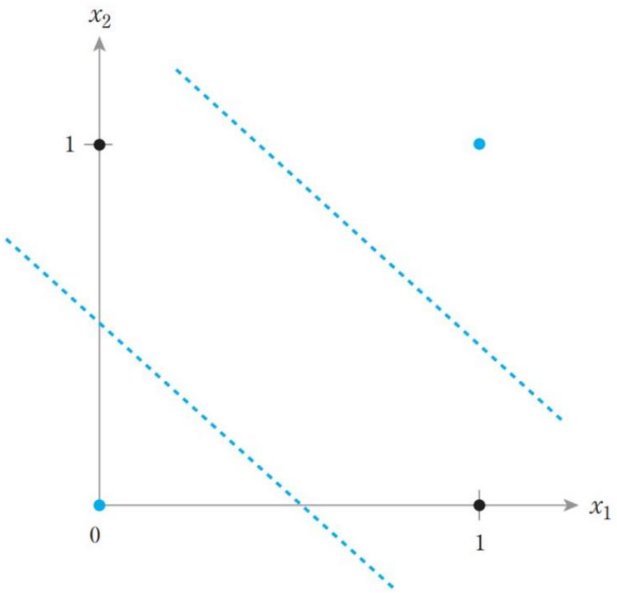
[그림 11-8] 퍼셉트론 파라미터의 설정

The perceptron is Linear classification (linear classification) Perform

Machine Translated by Google

Examples of cases where a single perceptron is impossible

- XOR logical operation example
  - 1 = If it is 2, it must be predicted as 0, and if it is 2, it must be predicted as 1 1
  - At least two straight lines are required for classification
- Cannot be classified as a single perceptron
  - Classification is possible by configuring a perceptron with two layers, but there is no training method.

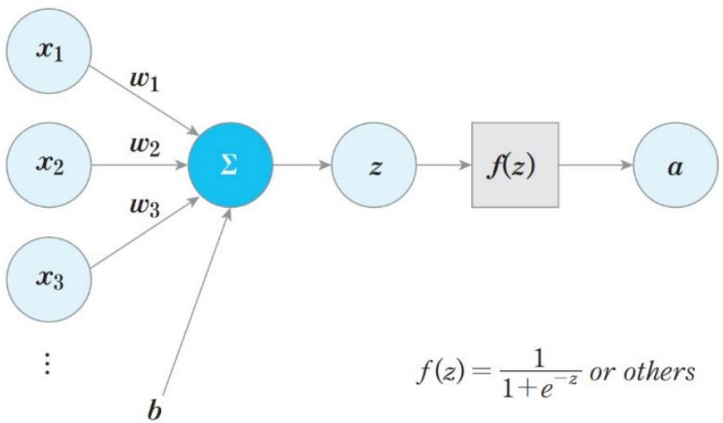


[그림 11–9] 한 개의 퍼셉트론으로 해결할 수 없는 분류의 예

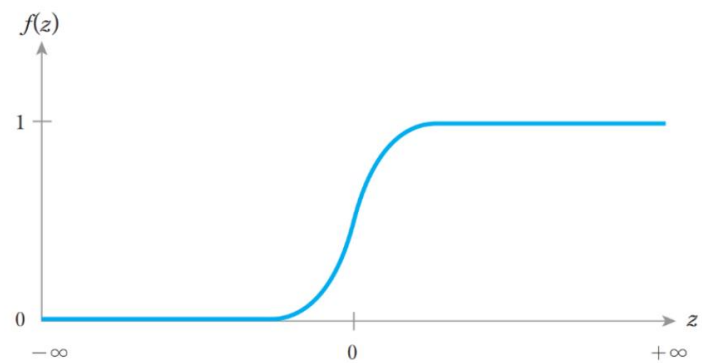
Machine Translated by Google

Modern neuron model

- Improved activation function to enable training of multi-layer neural networks
  - It has the same structure as a perceptron, but uses the sigmoid function as the activation function.
    - Regardless of the value of , 0 <(<)1 becomes



[그림 11-10] 현대적인 뉴런의 구조



[그림 11-11] 시그모이드 활성화 함수

- Prediction Calculation Process
  - Calculate weighted sum
  - z – Calculate a by applying an activation function to z
  - Output a prediction of 1 if the value of a is 0.5 or greater, otherwise output 0

$$z = w_1x_1 + w_2x_2 + w_3x_3 + \cdots + b$$

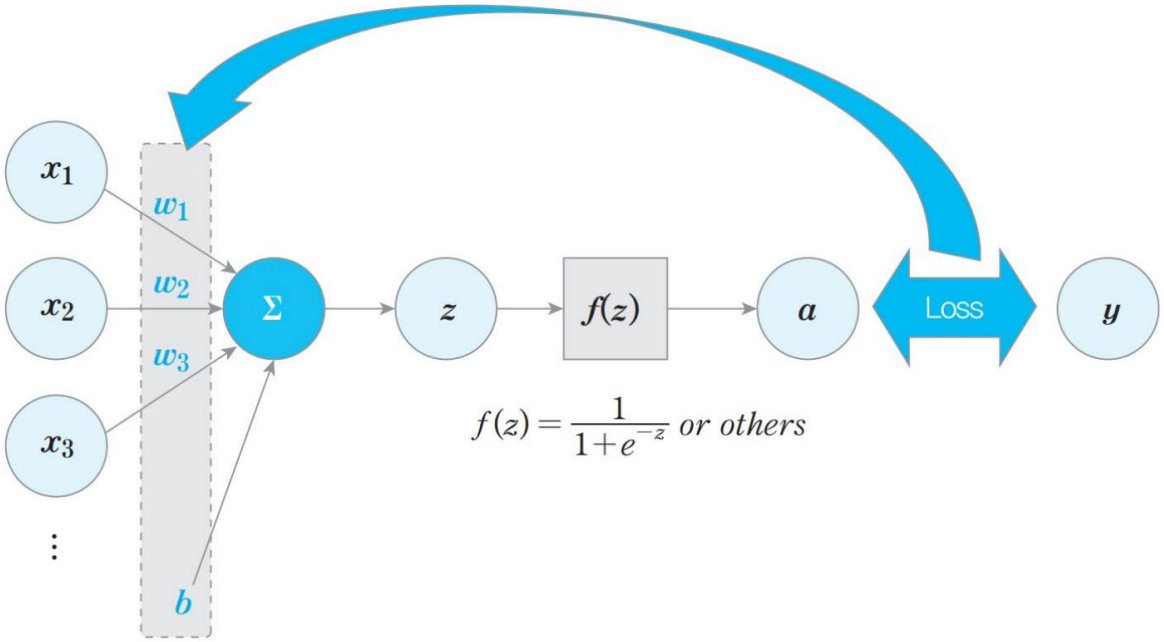
$$a = f(z) = \frac{1}{1+e^{-z}}$$

$$\hat{y} = \begin{cases} 1, & \text{if } a \geqq 0.5 \\ 0, & \text{if } a < 0.5 \end{cases}$$

Machine Translated by Google

Error backpropagation in a single neuron

- Concept of Error Backpropagation – Calculates the loss function using the error between the correct answer and the predicted value
  – Adjusts parameters (1, 2, … ,) in a direction that reduces the loss function



[그림 11-12] 오차 역전파 과정

Machine Translated by Google

Backpropagation of Error in a Single Neuron (Continued)

- The loss function utilizes entropy

$$L(w_1, w_2, w_3, \cdots, b) = -\frac{1}{n}\sum_{i=1}^{n}(y_i \log(a_i) + (1-y_i)\log(1-a_i))$$

- Calculate the gradient of the loss function and use it to modify parameters (chain differentiation)
  - The gradient is calculated through chain differentiation and is ultimately derived as a simple formula.

- ——— = ÿ ——— ——— ÿ = ——— $\frac{1}{\text{ÿ}}$ =1 ( ) ,

• = ÿ ——— ——— ——— ÿ = ——— $\frac{1}{\text{ÿ}}$ =1 ( )

Machine Translated by Google

Multilayer Artificial Neural Networks and Backpropagation Algorithms

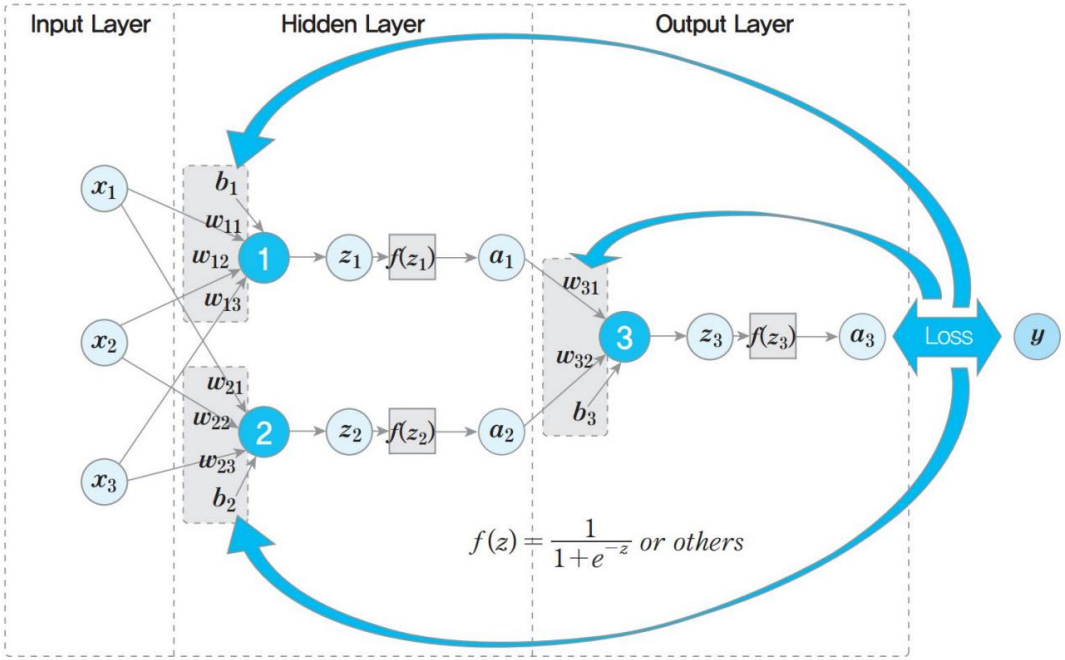
• Classification of

Layers – Input Layer: The layer containing input values 1, 2, and 3

– Output Layer: The layer containing the output neuron (neuron 3) –

Hidden Layer: The layer excluding the input and output (neurons 1 and 2) •

Since the input layer is excluded when counting the number of layers, the figure below is a 2-layer neural network.



[그림 11–13] 다계층 신경망의 오차 역전파

Machine Translated by Google

Multilayer Artificial Neural Networks and Backpropagation Algorithm (Continued

- Calculation of predicted values
  - Predicted values from neurons 1 and 2, respectively Calculate 1 and 2

$$a_1 = \frac{1}{1+e^{-z_1}}, \quad \text{여기서} \quad z_1 = w_{11}x_1 + w_{12}x_2 + w_{13}x_3 + b_1$$

$$a_2 = \frac{1}{1+e^{-z_2}}, \quad \text{여기서} \quad z_2 = w_{21}x_1 + w_{22}x_2 + w_{23}x_3 + b_2$$

  - Neuron 3 is Calculate 3 using 1 and 2 as input values

$$a_3 = \frac{1}{1+e^{-z_3}}, \quad \text{여기서} \quad z_3 = w_{31}a_1 + w_{32}a_2 + b_3$$

  - The final output value is determined as 1 if 3 is 0.5 or greater, and 0 otherwise.

- Loss function
  - Utilizing the entropy function, just like a single neuron

$$L = -\frac{1}{n}\sum_{i=1}^{n}\{y(i)\log(a_3(i)) + (1-y(i))\log(1-a_3(i))\}$$

Machine Translated by Google

Multilayer Artificial Neural Networks and Backpropagation Algorithm (Continued

- Backpropagation algorithm
  - Method for calculating gradients in multilayer artificial neural networks

  - The gradient of the output layer (neuron 3) is calculated in the same way as a single neuron.

    - $\frac{\quad}{31} = \frac{\quad}{3} \cdot \frac{3}{3} \cdot \frac{3}{31} = \frac{1}{\ddot{y}} \quad =1 \; 3 \left(\quad ( ) \cdot \quad ( ) \right) \; 1( )$
    - $\frac{\quad}{32} = \frac{\quad}{3} \cdot \frac{3}{3} \cdot \frac{3}{32} = \frac{1}{\ddot{y}} \quad =1 \; 3 \left(\quad ( ) \cdot \quad ( ) \right) \; 2( )$
    - $\frac{\quad}{31} = \frac{\quad}{3} \cdot \frac{3}{3} \cdot \frac{3}{\quad} = \frac{1}{\ddot{y}} \quad =1 \; 3 \left(\quad ( ) \cdot \quad ( ) \right)$

  - The gradient of the hidden layer (neuron 2) has a longer chain derivative, but the final calculation result is

    Represented as a simple product of expressions

    - $\frac{\quad}{1} = \frac{\quad}{3} \cdot \frac{3}{3} \cdot \frac{3}{1} \cdot \frac{1}{1} \cdot \frac{1}{1} = \frac{1}{\ddot{y}} \quad =1 \; 3 \left(\quad ( ) \cdot \quad ( ) \right) \; 31 \; 1 \; ( ) \left(1 \; \ddot{y} \quad 1( ) \right) \; ( )$
    - $\frac{\quad}{1} = \frac{\quad}{3} \cdot \frac{3}{3} \cdot \frac{3}{1} \cdot \frac{1}{1} \cdot \frac{1}{1} = \frac{1}{\ddot{y}} \quad =1 \; 3 \left(\quad ( ) \cdot \quad ( ) \right) \; 31 \; 1 \; ( ) \left(1 \; \ddot{y} \quad 1( ) \right)$
    - $\frac{\quad}{21} = \frac{\quad}{3} \cdot \frac{3}{3} \cdot \frac{3}{2} \cdot \frac{2}{2} \cdot \frac{2}{2} = \frac{1}{\ddot{y}} \quad =1 \; 3 \left(\quad ( ) \cdot \quad ( ) \right) \; 32 \; 2 \; ( ) \left(1 \; \ddot{y} \quad 2( ) \right) \; ( )$
    - $\frac{\quad}{2} = \frac{\quad}{3} \cdot \frac{3}{3} \cdot \frac{3}{2} \cdot \frac{2}{2} \cdot \frac{2}{2} = \frac{1}{\ddot{y}} \quad =1 \; 3 \left(\quad ( ) \cdot \quad ( ) \right) \; 32 \; 2 \; ( ) \left(1 \; \ddot{y} \quad 2( ) \right)$

Machine Translated by Google

Backpropagation example data

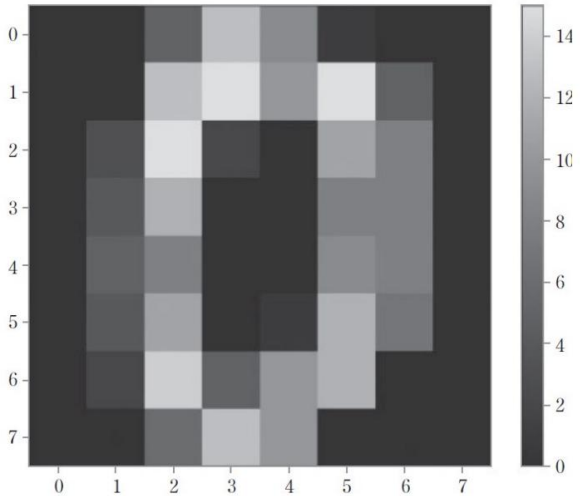
- Read data: Low-resolution (8x8) handwriting data

```
from sklearn import datasets
dataset = datasets.load_digits()
x_data = dataset.data
y_data = dataset.target

print('X shape = ', x_data.shape)
print('Y shape = ', y_data.shape)
```

```
X shape =  (1797, 64)
Y shape =  (1797,)
```

- Data Visualization: Example of Data 0

```
print('First x data = ', x_data[0])
print('First y data = ', y_data[0])

import matplotlib.pyplot as plt
plt.imshow(x_data[0].reshape(8,8))
plt.colorbar()
plt.show()
```



Machine Translated by Google

Data Preprocessing

- Data processing

```
1  import numpy as np
2  y_data = (y_data % 2).astype(np.float32)
3  x_data = (x_data / 15.0).astype(np.float32)
```

– (Line 2) Process the answer to distinguish only between even and odd numbers • This allows for only one output neuron – (Line 3) Process the input value to be between 0 and 1

- Split into training and test data

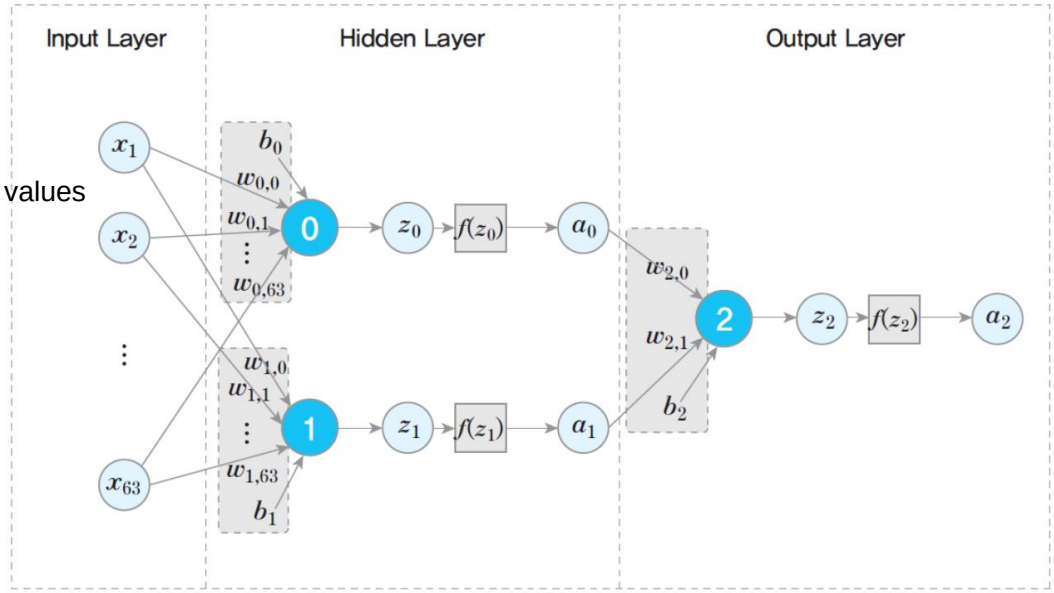
```
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x_data, y_data, test_size=0.2)
```

– Use 20% as test data

Machine Translated by Google

Implementation of Backpropagation

- Structure of the neural network to be implemented – Output layer neuron: 2 – Hidden layer neuron: 0, 1 – Input layer: 64 pixel brightness values



[그림 11-16] 구현하려는 인공신경망 구조

- Initialize all parameters to 0

```
w0 = np.zeros(64)
b0 = 0
w1 = np.zeros(64)
b1 = 0
w2 = np.zeros(2)
b2 = 0
```

Machine Translated by Google

Implementation of Backpropagation (continued)

- Function to calculate the output of a single neuron –

  (Lines 3–4) Calculate weighted

  sum z – (Line 5) Activation function

```
1  def output(data, w, b):
2      z = b
3      for j in range(len(data)):
4          z += w[j]*data[j]
5      a = 1/(1+np.exp(-z))
6      return a
```

- Function to calculate the output of the entire neural network –

  (Lines 6–7) Hidden layer calculation

  – (Line 8) Output layer calculation

```
1  def predict(x, w0, b0, w1, b1, w2, b2):
2      n = len(x)
3      a = np.zeros(n)
4      for i in range(n):
5          a_hid = np.zeros(2)
6          a_hid[0] = output(x[i], w0, b0)
7          a_hid[1] = output(x[i], w1, b1)
8          a_out = output(a_hid, w2, b2)
9          a[i] = a_out
10     return a
```

- Loss function: Use entropy function

```
1  def loss(a, y):
2      l = y*np.log(a) + (1-y)*(np.log(1-a))
3      return -np.mean(l)
```

Machine Translated by Google

Implementation of Backpropagation (continued)

- Gradient calculation
  - (Lines 21–22, 36) $\frac{\partial L}{\partial w_{2,j}} = \frac{1}{n}\sum_{i=0}^{n-1}(a_2(i)-y(i))a_j(i)$ calculate
  - (Lines 23, 37) $\frac{\partial L}{\partial b_2} = \frac{1}{n}\sum_{i=0}^{n-1}(a_2(i)-y(i))$ calculate
  - (Lines 25–27, 32–33) Calculate gradient for 0, 0

$$\frac{\partial L}{\partial w_{0,j}} = \frac{1}{n}\sum_{i=0}^{n-1}(a_2(i)-y(i))w_{2,0}(a_0(i)\,(1-a_0(i)))x_j(i)$$

$$\frac{\partial L}{\partial b_0} = \frac{1}{n}\sum_{i=0}^{n-1}(a_2(i)-y(i))w_{2,0}(a_0(i)\,(1-a_0(i)))$$

  - (Lines 29–31, 34–35) Calculate gradient for 1, 1

$$\frac{\partial L}{\partial w_{1,j}} = \frac{1}{n}\sum_{i=0}^{n-1}(a_2(i)-y(i))w_{2,1}(a_1(i)\,(1-a_1(i)))x_j(i)$$

$$\frac{\partial L}{\partial b_1} = \frac{1}{n}\sum_{i=0}^{n-1}(a_2(i)-y(i))w_{2,1}(a_1(i)\,(1-a_1(i)))$$

```
12      for i in range(n):
13          x = x_train[i]
14          y = y_train[i]
15
16          a_hid = np.zeros(2)
17          a_hid[0] = output(x, w0, b0)
18          a_hid[1] = output(x, w1, b1)
19          a_out = output(a_hid, w2, b2)
20
21          for j in range(2):
22              dw2[j] += (a_out-y)*a_hid[j]
23          db2 += (a_out-y)
24
25          for j in range(64):
26              dw0[j] += (a_out-y)*w2[0]*a_hid[0]*(1-a_hid[0])*x[j]
27          db0 += (a_out-y)*w2[0]*a_hid[0]*(1-a_hid[0])
28
29          for j in range(64):
30              dw1[j] += (a_out-y)*w2[1]*a_hid[1]*(1-a_hid[1])*x[j]
31          db1 += (a_out-y)*w2[1]*a_hid[1]*(1-a_hid[1])
32      dw0/=n
33      db0/=n
34      dw1/=n
35      db1/=n
36      dw2/=n
37      db2/=n
```

Machine Translated by Google

Implementation of Backpropagation (continued)

- Full code for the backpropagation section

```
eta = 0.1
for epoch in range(2000):
    a = predict(x_train, w0,b0,w1,b1,w2,b2)
    print('epoch', epoch, 'loss = ', loss(a, y_train))

    dw0 = np.zeros(64)
    db0 = 0
    dw1 = np.zeros(64)
    db1 = 0
    dw2 = np.zeros(2)
    db2 = 0
    for i in range(n):
        x = x_train[i]
        y = y_train[i]

        a_hid = np.zeros(2)
        a_hid[0] = output(x, w0, b0)
        a_hid[1] = output(x, w1, b1)
        a_out = output(a_hid, w2, b2)

        for j in range(2):
            dw2[j] += (a_out-y)*a_hid[j]
        db2 += (a_out-y)

        for j in range(64):
            dw0[j] += (a_out-y)*w2[0]*a_hid[0]*(1-a_hid[0])*x[j]
        db0 += (a_out-y)*w2[0]*a_hid[0]*(1-a_hid[0])

        for j in range(64):
            dw1[j] += (a_out-y)*w2[1]*a_hid[1]*(1-a_hid[1])*x[j]
        db1 += (a_out-y)*w2[1]*a_hid[1]*(1-a_hid[1])
    dw0/=n
    db0/=n
    dw1/=n
    db1/=n
    dw2/=n
    db2/=n

    w0 -= eta*dw0
    b0 -= eta*db0
    w1 -= eta*dw1
    b1 -= eta*db1
    w2 -= eta*dw2
    b2 -= eta*db2
```

For the full execution code, refer to ex11-4.py.

Machine Translated by Google

Prediction and performance metrics

- (Lines 94–98) Perform prediction –

  Predict 1 if the output value is 0.5 or greater,

  otherwise 0.

```
94  predictions = predict(x_test, w0,b0,w1,b1,w2,b2)
95  predLabels=[]
96  for y in predictions:
97      if y<0.5: predLabels.append(0)
98      else: predLabels.append(1)
99
100  from sklearn.metrics import confusion_matrix
101  from sklearn.metrics import classification_report
102  print(confusion_matrix(y_test, predLabels))
103  print(classification_report(y_test, predLabels))
```

- (Lines 100–103) Output performance metrics –

  Confirms good prediction performance with 93% accuracy

```
[[171  13]
 [ 14 162]]
              precision    recall  f1-score   support

         0.0       0.92      0.93      0.93       184
         1.0       0.93      0.92      0.92       176

    accuracy                           0.93       360
   macro avg       0.93      0.92      0.92       360
weighted avg       0.93      0.93      0.92       360
```

For the full execution code, refer to ex11-5.py.